

Project Seminar: Redundancy in Large Language Models

Praktikum / Project Seminar

Summer Semester 2026

1 Overview

Transformer-based language models are highly over-parameterized. This seminar studies redundancy in large language models (LLMs) at different granularities. Each group will focus on one granularity and investigate how redundancy can be measured, where it appears, whether it is removable, and whether the model can recover after removal or modification.

The five project groups are:

Group 1: Block / Layer-level redundancy

Group 2: Attention head-level redundancy

Group 3: Weight-level redundancy

Group 4: Tile / block-sparse-level redundancy

Group 5: Neuron / FFN-unit-level redundancy

The goal is not only to compress a model, but to understand what kind of redundancy exists at each granularity. In particular, each group should ask:

- How can redundancy be defined and measured at this granularity?
- Where does redundancy appear in the model?
- Is redundancy related to model depth, component type, model family, model size, training data, or downstream task?
- Can redundant units be removed with little performance degradation?
- If performance drops, can it be recovered by short fine-tuning?
- Are apparently redundant units interchangeable, mergeable, or shareable?

Some of these questions may not apply equally to all granularities. For example, block-level experiments cannot directly compare attention and FFN components inside a block unless the block is decomposed further. Such limitations should be explicitly discussed in the group reports.

2 Common Software and Evaluation Setup

The first part of the seminar is shared across all groups. Every group should be able to run a basic LLM evaluation pipeline before starting granularity-specific experiments.

2.1 Recommended Software Stack

All groups are encouraged to use the following tools:

- **uv** for Python environment and dependency management
- **PyTorch** for model execution and interventions
- **Hugging Face Transformers** for loading models and tokenizers
- **Hugging Face Datasets** for loading benchmark datasets
- **lm-evaluation-harness** for standardized evaluation
- **NumPy, SciPy, scikit-learn** for numerical analysis
- **Matplotlib / Seaborn** for plotting results
- Optional: **nnsight**, **Accelerate**, **Weights & Biases**, or **TensorBoard**

The exact model that each group can run will depend on the available hardware. Groups should first start with small models and only scale up if their hardware allows it.

2.2 Suggested Model Families

Groups may consider models such as:

- gpt2
- Qwen3
- Llama2

Groups should justify their model choice in their first report. A smaller model with careful analysis is better than a larger model with incomplete experiments.

2.3 Suggested Evaluation Tasks

Possible evaluation tasks include:

- Language modeling perplexity on WikiText-2 or a small C4 subset
- lm-evaluation-harness tasks such as PIQA, ARC-Easy, or HellaSwag subsets
- other topic related probing or synthetic tasks

3 Learning Resources

Students who are less familiar with LLM implementation details may use the following resources during the preparation phase:

- Stanford CS336: Language Modeling from Scratch
<https://stanford-cs336.github.io/spring2025/>
- Andrej Karpathy's nanoGPT project
<https://github.com/karpathy/nanoGPT>

- Hugging Face Transformers documentation
<https://huggingface.co/docs/transformers/index>
- lm-evaluation-harness
<https://github.com/EleutherAI/lm-evaluation-harness>
- The Transformer architecture and standard components: attention, MLP/FFN, residual stream, layer normalization, embeddings

These resources are not mandatory reading in full. They are intended to help students understand the software and model internals needed for the project.

4 Repository and Collaboration

All groups will collaborate in a shared Git repository. The repository should contain common infrastructure for model loading, evaluation, result logging, plotting, and group-specific measurements.

A possible repository structure is:

```
llm-redundancy-seminar/
  pyproject.toml
  README.md
  uv.lock

  configs/
    models/
    datasets/
    experiments/

  src/
    redundancy/
      models.py
      data.py
      eval.py
      hooks.py
      metrics/
      pruning/
      recovery.py
      plotting.py

  scripts/
    run_baseline.py
    run_measurement.py
    run_pruning.py
    run_recovery.py

  notebooks/
    group_x/

  reports/
    group_x/
```

Each group should work on its own branch or feature branches and submit changes through pull requests. Code should be reproducible: every result should include the model, dataset, metric, random seed, intervention type, and evaluation command.

5 Common Timeline

The exact weekly schedule can be adjusted, but the seminar should follow a common-to-specific structure: first build a shared pipeline, then develop group-specific measurements, then perform interventions and analysis.

Weeks 1–2: Software Pipeline and Baseline Evaluation

All groups should:

- Clone the shared Git repository.
- Set up the Python environment with `uv`.
- Load at least one Hugging Face causal language model.
- Load at least one dataset.
- Run a baseline evaluation using either a custom perplexity script or `lm-evaluation-harness`.
- Save results in a structured format, e.g. JSON.

Expected deliverable:

- A working baseline pipeline in the shared repository.
- A short note describing the model, dataset, evaluation command, and baseline result.

Weeks 3–4: Background Reading and Project Proposal

Each group should read one or two starting papers assigned to its granularity and search for additional related work. The purpose is to understand existing definitions, measurements, and interventions for redundancy.

Each group should prepare a one-page proposal answering:

1. What does redundancy mean at your granularity?
2. Which papers or methods are most relevant?
3. Which model(s) can you realistically run on your hardware?
4. Which dataset(s) and evaluation metric(s) will you use?
5. How might redundancy depend on depth, component type, model family, model size, or task?
6. How do you plan to measure redundancy?
7. What intervention will you try first: pruning, masking, replacement, merging, sharing, or fine-tuning recovery?
8. Which limitations do you expect at your granularity?

Expected deliverable:

- A one-page group proposal.
- A short presentation of the starting papers and planned experiment.

Weeks 5–8: Redundancy Measurement

Each group implements its own measurement method. At this stage, the focus is measurement and visualization, not yet compression.

Possible outputs include:

- Layer-wise, head-wise, neuron-wise, tile-wise, or component-wise redundancy scores
- Heatmaps
- Similarity matrices
- Activation statistics
- Weight distributions
- Singular value or norm distributions

Expected deliverable:

- First implementation of the group’s redundancy measurement.
- At least one plot or table showing where redundancy appears.

Weeks 9–10: Intervention and Evaluation

Each group applies an intervention based on its redundancy measurement.

Examples include:

- Removing or skipping blocks
- Masking attention heads
- Pruning individual weights
- Zeroing weight tiles
- Removing or merging FFN neurons

Each group should compare its redundancy-guided intervention against a random baseline at the same removal ratio.

Expected deliverable:

- Performance versus removal-ratio curve.
- Comparison with a random baseline.
- Discussion of whether the measured redundancy is actually removable.

Weeks 11–12: Recovery, Interchangeability, and Additional Dimensions

Groups should extend their experiments along one or more shared dimensions:

- **Depth:** shallow, middle, and deep layers
- **Component:** attention vs FFN; Q/K/V/O vs up/gate/down, where applicable
- **Model:** different model families or sizes, where hardware allows

- **Task:** language modeling vs downstream task evaluation
- **Recovery:** short fine-tuning after pruning or modification
- **Interchangeability:** swapping, copying, averaging, merging, or sharing units

Not every dimension applies to every group. For example, block-level redundancy may not have a component dimension, while weight-level redundancy may not have an obvious unit-interchangeability experiment. Groups should explicitly state which dimensions are applicable and which are not.

Expected deliverable:

- Extended experiment results.
- Analysis of at least one additional dimension beyond the first measurement.

Weeks 13–16: Final Analysis and Presentation

Each group prepares its final analysis and presentation. The final presentation should compare measurement, removability, recovery, and limitations.

Expected deliverable:

- Final slides.
- Cleaned code and reproducibility instructions.
- Final result files and figures.
- A short written report or contribution to a combined seminar report.

6 Group-Specific Starting Points

The papers below are starting points only. Each group is expected to find additional papers and decide which methods are most appropriate for their own experiments.

6.1 Group 1: Block / Layer-level Redundancy

Starting papers:

- Xin Men et al. *ShortGPT: Layers in Large Language Models are More Redundant Than You Expect*. 2024.
<https://arxiv.org/abs/2403.03853>
- Angela Fan et al. *Reducing Transformer Depth on Demand with Structured Dropout*. 2019.
<https://arxiv.org/abs/1909.11556>

Possible measurement directions:

- Similarity between layer hidden states
- CKA or cosine similarity across layers
- Norm of residual updates
- Layer sensitivity under skipping or removal

Possible interventions:

- Skip or remove selected layers
- Compare middle-layer pruning with shallow/deep-layer pruning
- Replace one layer with another layer(non-/ neighbouring)
- Fine-tune after removing layers

6.2 Group 2: Attention Head-level Redundancy

Starting papers:

- Paul Michel, Omer Levy, and Graham Neubig. *Are Sixteen Heads Really Better than One?* 2019.
<https://arxiv.org/abs/1905.10650>
- Elena Voita et al. *Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned.* 2019.
<https://arxiv.org/abs/1905.09418>

Possible measurement directions:

- Similarity between attention maps
- Attention entropy
- Similarity between per-head outputs
- Head importance under masking

Possible interventions:

- Mask individual heads or groups of heads
- Remove highly similar heads
- Replace one head output with another
- Compare redundancy-guided head removal with random head removal

6.3 Group 3: Weight-level Redundancy

Starting papers:

- Elias Frantar and Dan Alistarh. *SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot.* 2023.
<https://arxiv.org/abs/2301.00774>
- Mingjie Sun et al. *A Simple and Effective Pruning Approach for Large Language Models.* 2023.
<https://arxiv.org/abs/2306.11695>

Possible measurement directions:

- Magnitude distribution of weights

- Component-wise sparsity sensitivity: Q, K, V, O, up, gate, down
- Activation-aware weight importance
- Layer-wise sensitivity to unstructured pruning

Possible interventions:

- Magnitude pruning
- Activation-aware pruning
- Global vs layer-wise pruning
- Fine-tuning recovery after pruning

6.4 Group 4: Tile / Block-sparse-level Redundancy

Starting papers:

- Aojun Zhou et al. *Learning N:M Fine-grained Structured Sparse Neural Networks From Scratch*. 2021.
<https://arxiv.org/abs/2102.04010>
- Trevor Gale, Erich Elsen, and Sara Hooker. *The State of Sparsity in Deep Neural Networks*. 2019.
<https://arxiv.org/abs/1902.09574>

Possible measurement directions:

- Tile norm or tile energy
- Tile-wise activation sensitivity
- Tile similarity or clustering
- Comparison of tile sizes, e.g. 8×8 , 16×16 , 32×32
- N:M sparsity patterns such as 2:4 or 4:8

Possible interventions:

- Zero low-importance tiles
- Compare tile pruning with unstructured pruning at the same sparsity
- Try N:M sparsity masks
- Replace similar tiles with shared representatives, if feasible

Important note: tile-level redundancy is especially interesting because it connects model redundancy with hardware-friendly compression. However, real speedup may require specialized kernels and may not be visible from parameter count alone.

6.5 Group 5: Neuron / FFN-unit-level Redundancy

Starting papers:

- Mor Geva et al. *Transformer Feed-Forward Layers Are Key-Value Memories*. 2021.
<https://arxiv.org/abs/2012.14913>
- Xinyin Ma, Gongfan Fang, and Xinchao Wang. *LLM-Pruner: On the Structural Pruning of Large Language Models*. 2023.
<https://arxiv.org/abs/2305.11627>
- Optional: Zonglin Li et al. *The Lazy Neuron Phenomenon: On Emergence of Activation Sparsity in Transformers*. 2022.
<https://arxiv.org/abs/2210.06313>

Possible measurement directions:

- Activation frequency of FFN neurons
- Activation magnitude distribution
- Correlation between neuron activation patterns
- Neuron importance under ablation
- Task-specific neuron activation

Possible interventions:

- Remove or mask low-activation neurons
- Remove highly correlated neurons
- Merge or replace similar neurons
- Fine-tune after neuron pruning

7 Expected Final Output

Each group should produce:

- A short literature summary
- A clear definition of redundancy at its granularity
- A working measurement implementation
- At least one intervention experiment
- A comparison against a random or simple baseline
- Clean code and reproducibility instructions
- A final presentation and short written report